

Plan de Trabajo Integrado — Food Store

TPI Programación 2 + TFI Bases de Datos I

Sistema: Food Store – Gestión de Pedidos de Comida (Consola · Colecciones en memoria)

Equipo: 3 integrantes

Materias: Programación 2 · Bases de Datos I

Aclaración importante: JDBC **no fue contenido de ninguna de las dos materias**. La consigna de Prog-2 usa almacenamiento **en memoria con Colecciones** (confirmado en la sección "Información del Proyecto" y en la rúbrica, que evalúa "Uso de Colecciones y Excepciones"). El TFI de BD-I trabaja con **SQL puro y procedimientos almacenados** en MySQL, sin código Java de conexión. Los dos entregables son independientes pero usan el mismo dominio (Food Store).

Nota de equipo (actualización): el equipo se redujo de 4 a **3 integrantes**. Las tareas que tenía asignadas el cuarto integrante (UML, README, Informe PDF de Prog-2, Etapas 4 y 5 de BD-I, y la compilación de la entrega final) se redistribuyeron entre los 3 miembros actuales — ver sección de responsabilidades más abajo.

Integrantes

Integrante	Nombre completo
Integrante 1	Alfredo Alexander Castillo Belisario
Integrante 2	Cesia Madelein Chihuan Castilla
Integrante 3	Rocío Marianela Agüero

Contenidos cursados que se aplican en este TPI

Programación 2

Unidad	Tema	Dónde se usa en el TPI
U3 – Introducción a POO	Clases, objetos, encapsulamiento, <code>private</code> , getters/setters	Todas las entidades (<code>Categoria</code> , <code>Producto</code> , <code>Usuario</code> , <code>Pedido</code> , <code>DetallePedido</code>)
U4 – POO avanzada	Constructores, <code>this</code> , sobrecarga, <code>static</code> , <code>toString</code>	Constructores de cada entidad; <code>toString()</code> para listar en consola

Unidad	Tema	Dónde se usa en el TPI
U5 – UML Básico	Diagramas de clases, asociaciones, multiplicidades (1:N, N:1)	Diagrama UML final del proyecto
U6 – Colecciones	ArrayList , enums, for-each , size() , get() , add()	Almacenamiento en memoria de todas las entidades; enums Rol , Estado , FormaPago
U7 – Herencia y Polimorfismo	extends , @Override , clase abstracta, polimorfismo	Clase abstracta Base (con toString() abstracto) → todas las entidades heredan de ella
U8 – Interfaces y Excepciones	interface , implements , throws , try-catch , extends RuntimeException	Interfaz Calculable ; paquete exception/ con excepciones propias

Bases de Datos I

Unidad	Tema	Dónde se usa en el TPI
U1 – Introducción a BD	Conceptos de BD relacional, SGBD, modelo de datos	Marco teórico del informe PDF
U2 – Modelo Conceptual	DER, entidades, atributos, cardinalidades	DER del sistema Food Store (Etapa 1 BD)
U3 – Modelo Relacional	Tablas, PK, FK, UNIQUE, traducción del DER a SQL	01_esquema.sql con todas las tablas y constraints
U4 – Diseño de BD	Normalización (1FN, 2FN, 3FN), desnormalización	Decisiones de diseño del esquema documentadas en el informe
U5 – Lenguaje SQL	DDL/DML: CREATE , INSERT , SELECT , UPDATE , DELETE ; GROUP BY , HAVING , JOIN (INNER, LEFT), subconsultas, vistas (VIEW)	02_catalogo.sql , 03_carga_masiva.sql , 05_consultas.sql , 06_vistas.sql
U6 – Gestión del SGBD	Índices (CREATE INDEX , SHOW INDEX), EXPLAIN , OPTIMIZE TABLE , ON DELETE RESTRICT/CASCADE , DDL avanzado	04_indices.sql , 05_explain.sql
U7 – Integridad y Transacciones	ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), START TRANSACTION , COMMIT , ROLLBACK , deadlock, niveles de aislamiento (READ COMMITTED, REPEATABLE READ)	08_transacciones.sql , 09_concurrencia_guiada.sql
U8 – Seguridad y Privacidad	Principio de mínimos privilegios, GRANT , vistas de seguridad, consultas parametrizadas (anti-inyección SQL), tríada CIA	07_seguridad.sql , procedimiento almacenado sin SQL dinámico

Contexto del proyecto

Los dos trabajos comparten el mismo dominio: el sistema **Food Store**. El TPI de Programación 2 entrega la aplicación Java con almacenamiento **en memoria (Colecciones)** — sin conexión a BD. El TFI de Bases de Datos I entrega la capa de base de datos MySQL que modela ese mismo dominio, con modelado, carga masiva, consultas avanzadas, seguridad y concurrencia — todo en **SQL puro y procedimientos almacenados**, sin código Java de conexión.

Son **dos entregables separados** (uno por materia) pero contruidos sobre el mismo modelo de negocio y con el mismo equipo.

Estado general (resumen)

Materia	Bloque	Estado
Programación 2	Código fuente completo (4 entidades, CRUD, excepciones, interfaz, servicios, menús)	✓ Completado
Programación 2	README.md	✓ Completado
Programación 2	Diagrama UML	☐ Pendiente — Rocío
Programación 2	Informe PDF	☐ Pendiente — Alfredo
Programación 2	Video demostrativo	☐ Pendiente — los 3
Bases de Datos I	Etapas 1 — Modelado (01_esquema.sql , 02_catalogo.sql)	✓ Completado
Bases de Datos I	Etapas 2 — Datos masivos (03_carga_masiva.sql , 04_indices.sql)	✓ Completado
Bases de Datos I	Etapas 3 — Consultas (05_consultas.sql , 05_explain.sql , 06_vistas.sql)	✓ Completado
Bases de Datos I	Etapas 4 — Seguridad (07_seguridad.sql)	☐ Pendiente — Alfredo
Bases de Datos I	Etapas 5 — Concurrencia (08_transacciones.sql , 09_concurrencia_guiada.sql)	✓ Completado
Bases de Datos I	Entrega final (PDF BD-I + Anexo IA + ZIP)	☐ Pendiente — Rocío

Distribución de responsabilidades

Cada integrante es responsable principal de su módulo. **Todos deben hacer commits** para cumplir el criterio de trabajo en equipo (Git — 5 pts). Como el equipo quedó en 3 personas, el video debe mostrar a **los 3** a cámara al inicio (ver nota en la sección de rúbrica más abajo sobre este punto).

Integrante 1 — Alfredo Alexander Castillo Belisario

Prog 2: U3 · U4 · U6 · U7 | **BD I:** U2 · U3 · U4 · U8

Programación 2 — Completado

- Arquitectura base del proyecto (paquetes, `Base`, enums, `Calculable`, excepciones).
- Épica 1 — Gestión de Categorías (`Categoria`, `CategoriaRepository`, `CategoriaService`, `MenuCategoria`), con validación de nombre único y aviso al eliminar una categoría con productos asociados.

Bases de Datos I — Etapa 1 (Modelado) — Completado

- DER, `01_esquema.sql`, `02_catalogo.sql`, inserts válidas/inválidas.

Pendiente — Informe PDF de Programación 2 (6-8 páginas)

Sección	Contenido
Carátula	Institución, carrera, materia, título, integrantes (los 3), fecha
Índice	Con números de página
Marco teórico	Java 21, POO, Colecciones, Interfaces, Excepciones (breve)
Decisiones técnicas	Arquitectura de paquetes (entities/enums/exception/interfaces/repository/service/menu), soft delete, por qué se descartó el login agregado en una iteración intermedia
Capturas	Los 4 submenús (Categorías, Productos, Usuarios, Pedidos), casos de éxito y de error
Dificultades	Encontradas y cómo se resolvieron (ej. stock con producto repetido en el mismo pedido)
Bibliografía	Herramientas usadas, incluyendo IA si se utilizó, con el modo de uso
Links	Repositorio + video + (UML de Rocío)

Pendiente — Bases de Datos I — Etapa 4: Seguridad e Integridad

Objetivo: aplicar mínimos privilegios, vistas protectoras y consultas seguras contra inyección SQL.

#	Tarea
1	Crear usuario MySQL con privilegios mínimos (<code>CREATE USER</code> , <code>GRANT SELECT</code> solo en tablas/vistas específicas); documentar prueba de acceso restringido
2	Crear 2 vistas protectoras: <code>vista_usuarios_publicos</code> (sin contraseña) + <code>vista_pedidos_sin_datos_personales</code> (sin mail ni celular)
3	Documentar ≥ 2 pruebas de integridad: duplicación de PK $\rightarrow$ error; violación de FK $\rightarrow$ error; valor fuera de CHECK $\rightarrow$ error
4	Implementar procedimiento almacenado sin SQL dinámico (anti-inyección SQL); documentar intento malicioso + resultado esperado + por qué queda neutralizado
5	Interactuar con IA para reflexionar sobre decisiones de seguridad $\rightarrow$ Anexo IA — Etapa 4

Entregables: `07_seguridad.sql` (usuario MySQL, GRANTS, 2 vistas, procedimiento almacenado), pruebas de integridad documentadas, script y documentación anti-inyección, Anexo IA — Etapa 4.

Integrante 2 — Cesia Madelein Chihuan Castilla

Prog 2: U3 · U4 · U6 · U7 | **BD I:** U3 · U5 · U6 · U7

Programación 2 — Completado

- Épica 2 — Gestión de Productos (`Producto` , `ProductoRepository` , `ProductoService` , `MenuProducto`), con validación de nombre/precio/stock.
- Épica 3 — Gestión de Usuarios (`Usuario` , `UsuarioRepository` , `UsuarioService` , `MenuUsuario`), con mail único.

Pendiente (de Prog 2) — `README.md` — Completado

- Requisitos previos, instrucciones de ejecución (NetBeans y terminal), estructura de paquetes, funcionalidades principales, y sección de enlaces a video/PDF/UML.

Bases de Datos I — Etapa 2 (Datos masivos) — Completado

- `03_carga_masiva.sql` , `04_indices.sql` , cuadro de verificaciones.

Bases de Datos I — Etapa 5: Concurrencia y Transacciones — Completado

Objetivo: demostrar el comportamiento de la BD ante accesos simultáneos.

#	Tarea
1	Simular ≥ 1 deadlock con 2 sesiones MySQL; documentar error 1213 / SQLSTATE 40001; guion en <code>09_concurrencia_guiada.sql</code>
2	Implementar transacciones en <code>08_transacciones.sql</code> : <code>START TRANSACTION</code> / <code>COMMIT</code> / <code>ROLLBACK</code> ; si deadlock 1213 $\rightarrow$ retry hasta 2 reintentos con backoff; logging de errores vía <code>SELECT</code>
3	Comparar READ COMMITTED vs REPEATABLE READ: ejemplo simple y observable de diferencia

#	Tarea
4	Escribir informe breve (5-10 líneas) con observaciones sobre concurrencia y transacciones
5	Interactuar con IA para interpretar resultados → Anexo IA — Etapa 5

Entregables: `08_transacciones.sql`, `09_concurrencia_guiada.sql`, deadlock documentado, comparativa de aislamiento, informe breve + Anexo IA — Etapa 5. Todo completo y verificado contra un MySQL 8.0 real (contenedor Docker descartable): detalle conceptual en `Etapas5_Descripcion_Conceptual.md` (mismo formato que `Etapas2_Descripcion_Conceptual.md`) y evidencia de IA en `Anexo_IA_Etapas5_FoodStore.docx`.

Integrante 3 — Rocío Marianela Agüero

Prog 2: U6 · U7 · U8 | **BD I:** U5 · U6

Programación 2 — Completado

- Épica 4 — Gestión de Pedidos y Detalles (`Pedido`, `DetallePedido`, `PedidoRepository`, `PedidoService`, `MenuPedido`), incluyendo `addDetallePedido`, `findeDetallePedidoByProducto`, `deleteDetallePedidoByProducto` y `calcularTotal()` vía `Calculable`.
- Paquete `exception/` (`EntidadNoEncontradaException`, `ValidationException`, `StockInvalidoException`).

Bases de Datos I — Etapa 3 (Consultas) — Completado

- `05_consultas.sql`, `05_explain.sql`, `06_vistas.sql`.

Pendiente — Diagrama UML final de Programación 2

Aspecto	Requisito
Herramienta	draw.io o Lucidchart
Contenido	Atributos, métodos, herencia de <code>Base</code> (con <code>toString()</code> abstracto), interfaz <code>Calculable</code> , enums, relaciones con multiplicidades
Crítico	Marcar explícitamente la relación unidireccional <code>Producto</code> → <code>Categoria</code> (<code>Producto</code> conoce su <code>Categoria</code> , pero <code>Categoria</code> NO tiene lista de <code>Productos</code>) — es el punto excluyente de la rúbrica
Entrega	Exportar como imagen e incluir en el informe (Alfredo) y en el repositorio

Pendiente — Bases de Datos I — Entrega final

Tarea	Detalle
Compilar PDF BD-I	Portada, resumen ejecutivo, reglas de negocio, DER, decisiones de diseño, evidencias de las 5 etapas (pedir material a Alfredo y Cesia de sus etapas), referencias cruzadas a los scripts

Tarea	Detalle
Compilar Anexo IA	Juntar el texto de chats con IA de las 5 etapas (de los 3 integrantes) en un único anexo, organizado por etapa
Nomenclatura	TFI_BDI_ComisionX_GrupoY_Apellidos.pdf y .zip
Empaquetado final BD-I	ZIP con los 9 scripts .sql numerados + README.txt con orden de ejecución y versión de MySQL
Empaquetado final Prog-2	ZIP con código fuente Java + PDF informe (coordinar con Alfredo una vez listo el informe)

Video demostrativo (responsabilidad compartida — los 3)

Parte	Descripción
Inicio obligatorio	Los 3 integrantes a cámara con nombre completo visible (sin excepción — si falta, la nota final es 0)
Duración	10 a 15 minutos
Resto	Puede continuar con cámara apagada, siempre con voz propia (no IA)
Demo Prog-2	CRUD completo de las 4 entidades + validaciones + mensajes de error
Demo BD-I	Consultas complejas, vistas, seguridad, concurrencia
Cierre	Conclusiones y aprendizajes
Acceso	Subir a plataforma con permisos públicos de visualización
Link	Agregarlo en el README.md y en ambos informes PDF

Nota sobre la rúbrica de Prog-2: la pauta de corrección describe el nivel "Excelente" (30/30) del criterio Video con la frase "4 a cámara", redactada pensando en equipos de 4. Con 3 integrantes, el nivel inmediatamente alcanzable según la letra de la rúbrica es el "Adecuado" (21/30), que explícitamente admite "3-4 a cámara". No es algo que se pueda evitar reduciendo el equipo a 3 — conviene mencionarlo con naturalidad si se pregunta en la corrección, dejando claro que los 3 integrantes reales del equipo están presentes desde el inicio.

Cronograma actualizado

Bloque	Actividad	Responsable	Estado
Prog-2	Setup, Base, enums, Calculable, Categorías	Alfredo	✓
Prog-2	Productos y Usuarios	Cesia	✓

Bloque	Actividad	Responsable	Estado
Prog-2	Pedidos, <code>DetallePedido</code> , <code>exception/</code>	Rocío	✓
Prog-2	Corrección de bugs (stock con producto repetido, nombre vacío, captura genérica) y alineación con la cursada	Sesión de revisión conjunta	✓
Prog-2	<code>README.md</code>	Cesia	✓
Prog-2	Diagrama UML	Rocío	☐
Prog-2	Informe PDF	Alfredo	☐
BD-I	Etapa 1 (Modelado)	Alfredo	✓
BD-I	Etapa 2 (Datos masivos)	Cesia	✓
BD-I	Etapa 3 (Consultas)	Rocío	✓
BD-I	Etapa 4 (Seguridad)	Alfredo	☐
BD-I	Etapa 5 (Concurrencia)	Cesia	✓
BD-I	Entrega final (PDF + Anexo IA + ZIP)	Rocío	☐
Ambas	Grabación del video (10-15 min, los 3 a cámara al inicio)	Todos	☐
Ambas	Empaquetado final: ZIP Prog-2 + ZIP BD-I + PDFs	Rocío (coordina)	☐

Rúbricas de evaluación

Programación 2 — Puntaje total: 100 pts

Criterio	Pts	Para obtener el máximo
Video	30	10-15 min, equipo completo a cámara, CRUD completo (A y B), explica clases/ interfaces/colecciones — <i>con 3 integrantes, el techo realista según la letra de la rúbrica es 21/30 ("3-4 a cámara")</i>
Correctitud funcional	20	CRUD completo, relación 1 → 1 unidireccional efectiva (A → B), unicidad, búsquedas correctas
Uso de Colecciones y Excepciones	10	ArrayList/colecciones eficientes, excepciones propias bien implementadas y capturadas
Arquitectura y Diseño	10	Separación clara: dominio / interfaces / lógica de negocio / menú; alta cohesión
Reproducibilidad	10	Repo público, README con requisitos + ejecución + link al video, compila a la primera
AppMenu & Errores	5	Menú claro, validaciones de entrada, mensajes éxito/error, casos borde contemplados
Calidad de código	5	Nombres significativos, <code>toString()</code> útiles, poca duplicación, comentarios justos

Criterio	Pts	Para obtener el máximo
Trabajo en equipo & Git	5	Commits repartidos entre los 3, mensajes claros, ramas/PRs o tag de entrega
UML coherente	3	Diagrama actualizado y consistente con clases/relaciones finales
Informe PDF	2	6-8 págs., claro, decisiones de diseño, capturas, fuentes/herramientas

Reglas excluyentes (*nota final = 0 si no se cumplen*):

Condición	Consecuencia
Sin video	Nota = 0
Repositorio no público	Nota = 0
No hay relación 1 → 1 real en el código	Nota = 0

Penalizaciones y bonus:

Sin baja lógica implementada	-3 pts
Plagio / copia	0 final
Logging útil + filtros de entrada robustos	+2 pts

Bases de Datos I — Entregables por etapa

Etapa	Responsable	Estado	Entregables principales
E1 Modelado	Alfredo	✓	DER + 01_esquema.sql + 02_catalogo.sql + inserts válidas/inválidas + Anexo IA
E2 Datos masivos	Cesia	✓	03_carga_masiva.sql + 04_indices.sql + descripción conceptual + cuadro verificaciones + Anexo IA
E3 Consultas	Rocío	✓	05_consultas.sql + 05_explain.sql + 06_vistas.sql + tabla tiempos + Anexo IA
E4 Seguridad	Alfredo	☐	07_seguridad.sql + pruebas integridad + anti-inyección + Anexo IA
E5 Concurrencia	Cesia	✓	08_transacciones.sql + 09_concurrencia_guiada.sql + informe + Anexo IA
General	Rocío (coordina)	☐	PDF BD-I + ZIP scripts + README.txt + Anexo IA en texto + video

Checklist de entrega final

Programación 2

- [X] Repositorio GitHub **público** con código fuente organizado por paquetes (*verificar visibilidad en GitHub antes de entregar*)
- [X] CRUD completo para Categoría, Producto, Usuario y Pedido
- [X] Soft delete en todas las entidades (`eliminado = true` , nunca remover de colección)
- [X] Relación 1 → 1 unidireccional en código: objeto A referencia a objeto B (no solo ids)
- [X] Colecciones (`ArrayList` , etc.) usadas eficientemente
- [X] Paquete `exception/` con excepciones propias creadas, lanzadas y capturadas (con multicatch específico)
- [X] Interfaz `Calculable` implementada en `Pedido` con `calcularTotal()`
- [X] AppMenu con validaciones de entrada robustas, mensajes claros de éxito y error
- [X] `Main` / AppMenu no contiene lógica de negocio compleja (delegada a services)
- [] Commits repartidos entre los 3 integrantes con mensajes descriptivos
- [] UML final coherente con el código (incluye relación 1 → 1 unidireccional) — **Rocío**
- [X] `README.md` con pasos claros + sección de links a video/PDF — **Cesia** (*completar los links cuando estén listos*)
- [] Informe PDF (6-8 págs.) con carátula, índice, marco teórico, capturas, decisiones — **Alfredo**
- [] Video 10-15 min: los 3 a cámara al inicio, CRUD demo, explicación diseño — **Todos**
- [] ZIP final: código fuente + PDF informe — **Rocío** (coordina)

Bases de Datos I

- [X] `01_esquema.sql` idempotente (`DROP IF EXISTS`) con PK/FK/UNIQUE/CHECK en todas las tablas
- [X] `02_catalogo.sql` con datos semilla
- [X] `03_carga_masiva.sql` con ≥10K registros coherentes y FKs válidas
- [X] `04_indices.sql` con índices para las consultas de Etapa 3
- [X] `05_consultas.sql` : 4 consultas (≥2 JOIN, 1 GROUP BY+HAVING, 1 subconsulta) con utilidad documentada
- [X] `05_explain.sql` : medición con/sin índice (mediana de 3 corridas, EXPLAIN, conclusión)
- [X] `06_vistas.sql` : ≥1 vista útil en el contexto del sistema
- [] `07_seguridad.sql` : usuario MySQL con mínimos privilegios + GRANTS + 2 vistas de seguridad — **Alfredo**
- [] Pruebas de integridad: ≥2 documentadas (PK duplicada, FK violada, CHECK violado) — **Alfredo**
- [] Consulta segura: procedimiento almacenado SQL sin SQL dinámico + prueba anti-inyección documentada — **Alfredo**
- [X] `08_transacciones.sql` : transacciones START/COMMIT/ROLLBACK + retry ante deadlock — **Cesia**
- [X] `09_concurrencia_guiada.sql` : simulación deadlock 2 sesiones + comparación aislamiento — **Cesia**

- [] README.txt : orden de ejecución de los 9 scripts + versión de MySQL — **Rocío**
- [x] DER imagen disponible (der_food_store.png) para incluir en PDF BD-I
- [] Anexo IA: texto completo de chats por Tema/Etapa (no capturas) — **Rocío compila, todos aportan**
- [] PDF BD-I completo: portada, resumen ejecutivo, reglas de negocio, DER, decisiones, evidencias por etapa, referencias cruzadas a scripts, Anexo IA, link al video — **Rocío**
- [] Video 10-15 min con link en el PDF BD-I (puede ser el mismo video que Prog-2 — coordinar) — **Todos**
- [] Nomenclatura correcta: TFI_BDI_ComisionX_GrupoY_Apellidos.pdf y .zip — **Rocío**
- [] ZIP BD-I contiene todos los scripts .sql + README.txt — **Rocío**